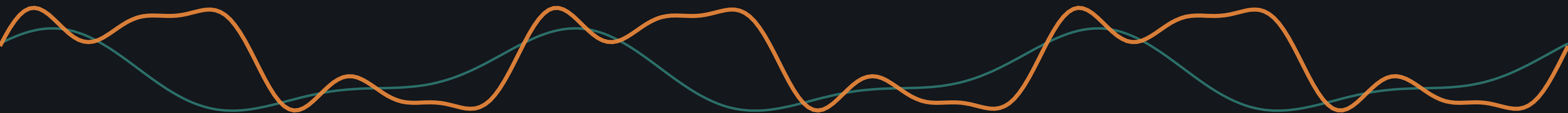


CSE 220 · SIGNALS AND LINEAR SYSTEMS · PROJECT

WaveKitchen

A cooking game where every kitchen action is a signal-processing operation

Kazi Asef Kabir · Tajrian Shams · [Team member] · [Team member]



PLAN

Who presents what

Part	Topic	Slides	Presented by
1	Motivation, idea and game loop	3 – 6	[Presenter]
2	Theory at every station	7 – 11	[Presenter]
3	Architecture and implementation	12 – 17	[Presenter]
4	Results, testing and live demo	18 – 21	[Presenter]

Every member can answer questions on the whole project, not only their part.

THE PROBLEM

Signals and Systems is taught as equations you cannot see or hear

Convolution

is a formula on the board, not
something you can hear.

Aliasing

is a warning in the textbook, not a
mistake you make.

Poles and zeros

are dots on a plane, not a filter you
tune.

Our goal: give every concept a visible, audible consequence, and a score that tells you whether you got it right.

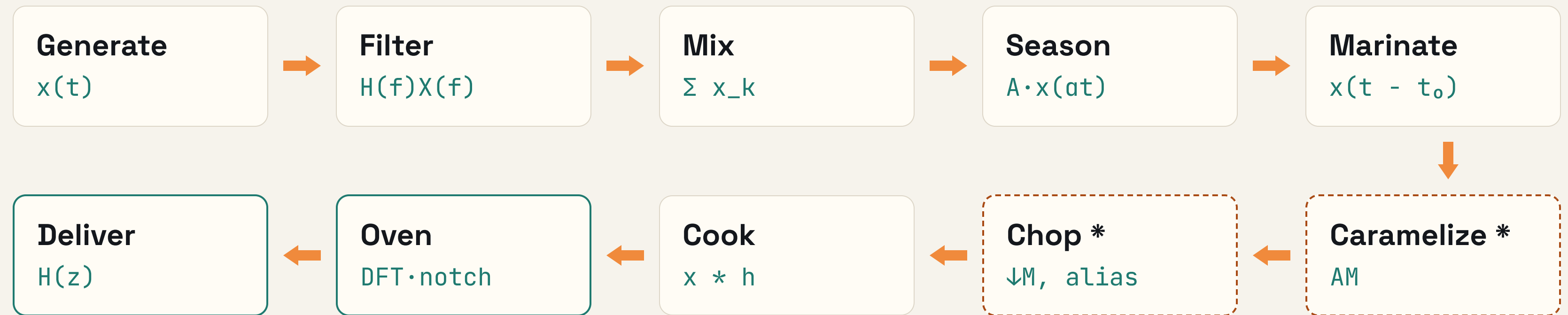
CORE IDEA

Every station is a system

Ingredient	$x(t)$	Dirty ingredient	$x(t) + n(t)$
Washing	$Y(f) = H(f) \cdot X(f)$	Mixing bowl	$\sum x_k(t)$
Seasoning	$A \cdot x(at)$	Marinating	$x(t - t_o)$
Cooking	$y = x * h$	Precision Oven	DFT $\rightarrow$ notch $\rightarrow$ IDFT
Delivery cart	$H(z)$: poles, zeros	Tasting	correlation $\cdot$ MSE $\cdot$ SNR

APPROACH

One dish, one signal chain



■ * only in some recipes (Crème Brûlée, Salad, Feast)

■ finishing stations: their output is the served dish

APPROACH

A timed run against a reference dish

- 1. Pick one of **10 recipes** across 4 tiers
- 2. Choose a difficulty: it sets the timer and the score multiplier
- 3. Stations unlock in order; each shows your signal against the target
- 4. Match the hidden **reference dish**: the same chain on clean ingredients with the ideal settings
- 5. Serve: score, stars, chef points and a leaderboard place

Difficulty		
Level	Time	Score
Easy	5:00	× 0.8
Medium	3:30	× 1.0
Hard	2:00	× 1.25
MasterChef	1:00	× 1.5

Time left becomes a bonus: 2 points per second, at most 300.

Ingredients, noise and washing

Ingredients are textbook signals

- **Square wave:** odd harmonics, $1/n$ (sugar, salt)
- **Triangle wave:** odd harmonics, $1/n^2$ (cheese)
- **Sums of sines:** bread = 3 Hz + 9 Hz
- **2-D parametric curves:** noodles, onion, egg
- **A real recording:** chicken, 44.1 kHz PCM

Washing is low-pass filtering

$$Y(f) = H(f) \cdot X(f)$$

- Noise arrives added: $x(t) + n(t)$
- Drag the cutoff across the spectrum
- Too low a cutoff cuts the ingredient itself
- Server prep score: 28 % of the dish score

Mix, season, marinate

Mix: superposition

$$y = (1/\sqrt{K}) \cdot \sum x_k$$

The bowl is linear: add the ingredients and scale by $1/\sqrt{K}$ to keep loudness comparable. Curves like noodles add as 2-D (x, y) paths.

Season: scaling

$$y = A \cdot x(\alpha t)$$

$$\leftrightarrow (A/|\alpha|) \cdot X(f/\alpha)$$

Two dials: amplitude A and rate α . $\alpha > 1$ compresses time and moves energy to higher frequencies.

Marinate: time shift

$$y = x(t - t_o)$$

$$\leftrightarrow X(f) \cdot e^{(-j2\pi f t_o)}$$

A pure delay. The server shifts by fractional samples with a linear phase ramp.

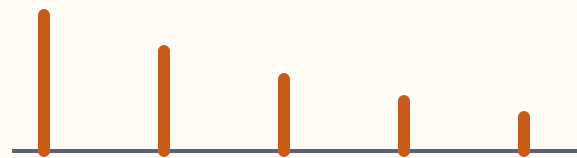
Crème Brûlée, Feast: amplitude modulation $x(t)[1 + m \cos 2\pi f_c t]$ adds sidebands at $f \pm f_c$.

Salad, Feast: decimation by M ; skip the anti-alias filter and content above $f_s/2M$ aliases.

Cooking is convolution

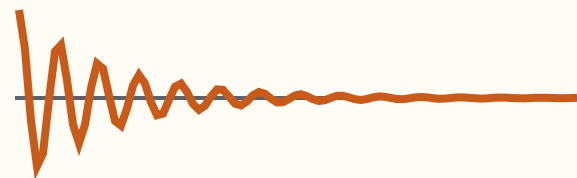
$$y[n] = \sum x[k] \cdot h[n - k]$$

An LTI system is fully described by its impulse response h . Each appliance is a different h ; the player slides h across x and watches the output build up.



Grill

A comb of echoes



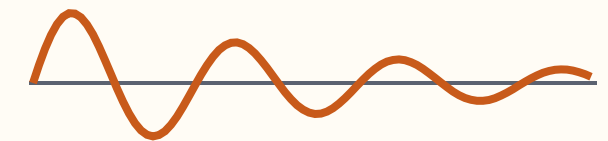
Fry

A bright, fast-decaying burst



Bake

A smooth long tail: a low-pass

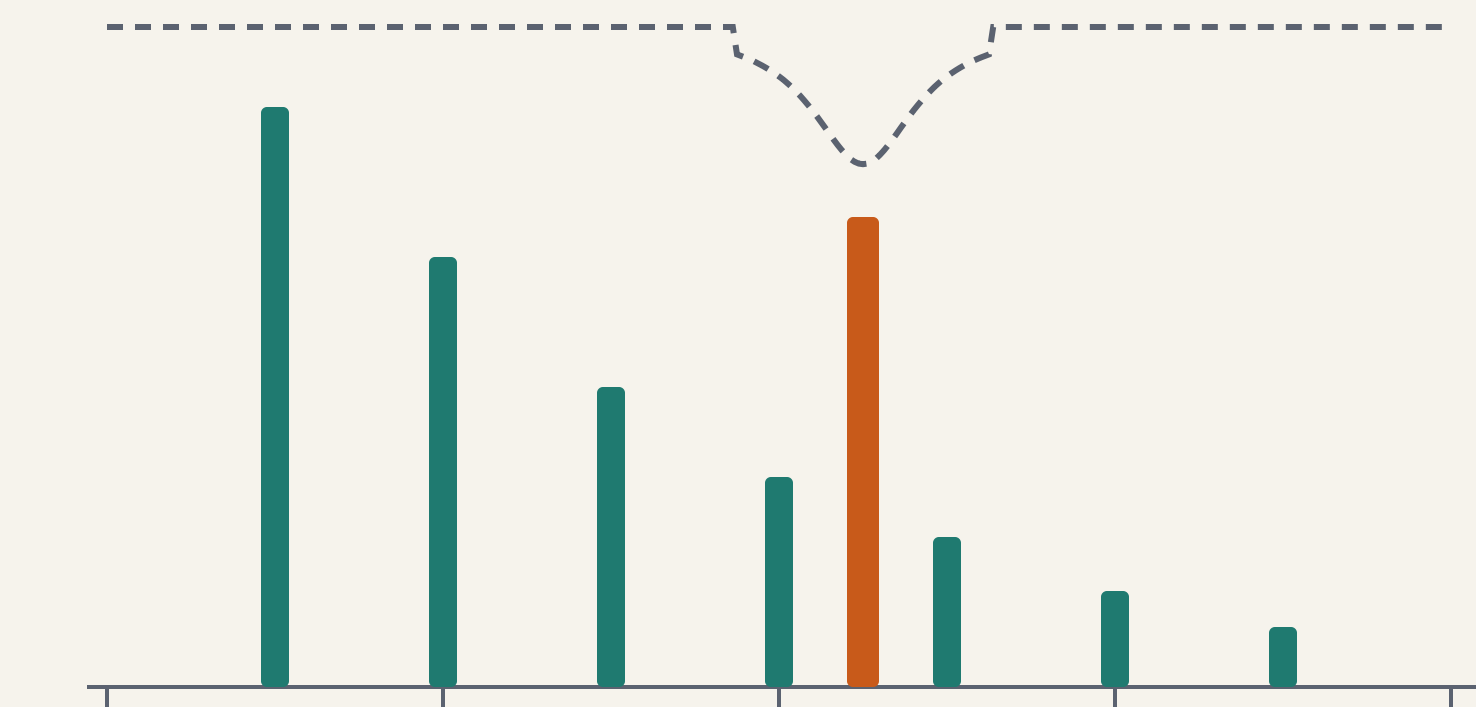


Boil

A damped resonator

Kernels: 96 taps with $\sum |h| = 1$, so cooking reshapes the dish without changing its level.

Precision Oven: remove the burnt overtone

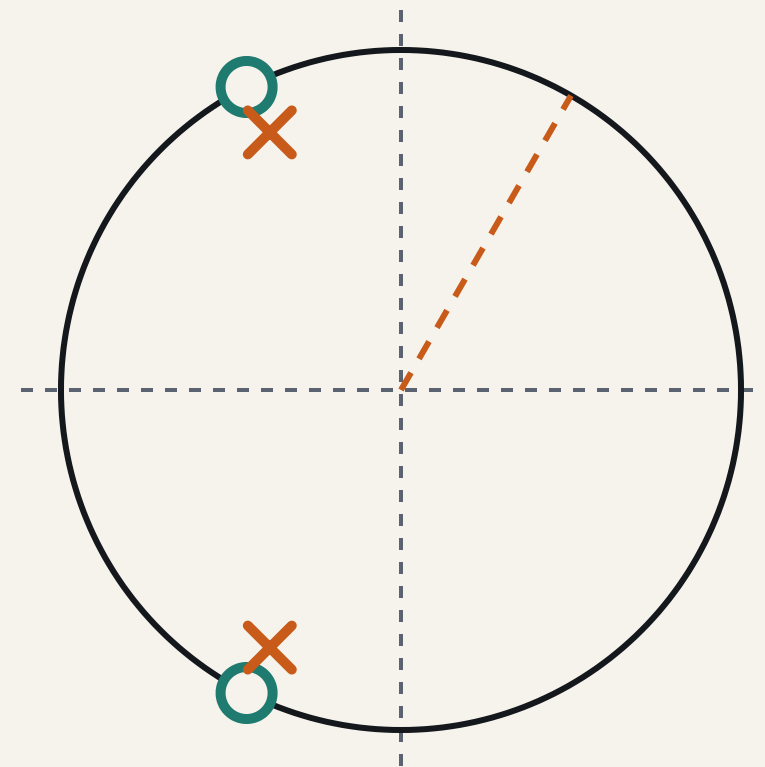


■ dish harmonics (every 4 Hz) ■ burnt overtone, 18 Hz

x-axis: 0 → 32 game Hz · dashed line: the notch filter gain

1. **Sample** at $f_s \geq 2 \cdot f_{\text{max}}$ (at most 64 Hz), or the dish aliases
2. **DFT**: bins at $f_k = k \cdot f_s / N$ (zero-padded radix-2 FFT)
3. **Spot the spike** the target does not have: it sits between two harmonics (burger: 18 Hz, between 16 and 20)
4. **Notch** it: gain $1 - 0.98 / (1 + (\Delta f / 1.5)^2)$
5. **Inverse DFT**: the rebuilt dish is what gets served

System Delivery on the z-plane



● zeros at $\pm\omega_0$

■ poles at $r = 0.85$

dashed: the aliased sensor reading

$$H(z) = g \cdot (z - e^{j\omega_0})(z - e^{-j\omega_0}) / (z - re^{j\omega_0})(z - re^{-j\omega_0})$$

- The road adds a tone: burger 2600 Hz at 8 kHz, so $\omega_0 = 2\pi \cdot 2600 / 8000 \approx 2.04$ rad
- Zeros on the unit circle at $\pm\omega_0$ remove it; poles at $r = 0.85$ keep the notch narrow
- g normalises the gain to 1 where the dish lives, so the dish passes untouched
- A pole with $|p| \geq 1$ is unstable (not BIBO): the output grows and the dish is ruined
- A 1.2 kHz sensor reports 2600 Hz as 200 Hz (aliased): a notch there misses

Two halves, one judge

Browser: plays

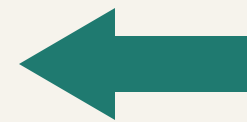
React 19 · TanStack Router/Start · TypeScript · Vite

- The DSP pipeline for every station
- Graphs and Web Audio playback
- Run state in localStorage
- Works offline with a local score

JSON over /api



settings: bowl, cutoffs,
dials, appliance, notch,
poles



scores, stages, ranks

Server: judges

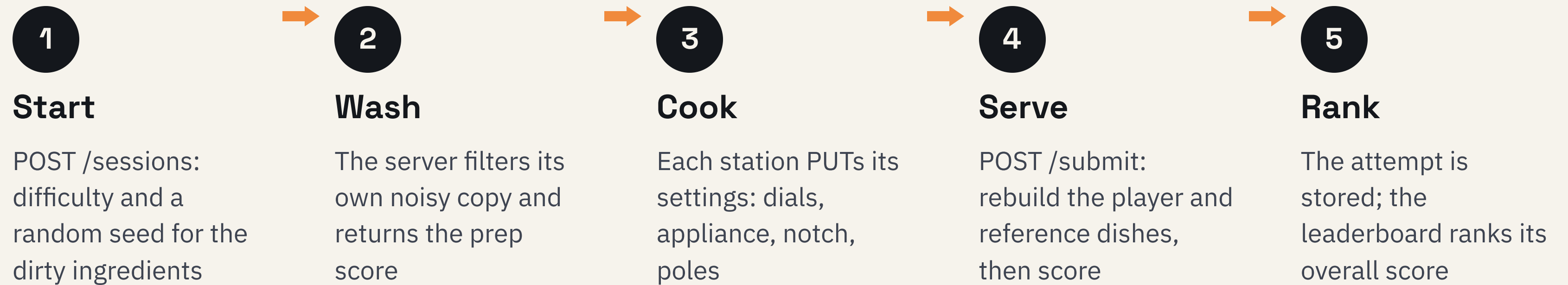
FastAPI · SQLAlchemy · SQLite · NumPy/SciPy

- Seeded dirty ingredients per session
- Rebuilds and scores the dish
- Points, ranks and tier unlocks
- The leaderboard

run.py starts both: FastAPI on port 8000, and Vite on 5173, which proxies /api to it.

ARCHITECTURE

A run, end to end



IMPLEMENTATION

How a signal lives in the game

401

samples per signal: one period, the last sample repeats the first

1 s

window, so frequencies are cycles per window ("game Hz")

110–880

Hz playback pitch: the window repeats until you can hear it

22,050

Hz: the server builds audio-rate dishes

The server maps game Hz to real Hz using where each dish holds 95 % of its energy, so a notch set in the browser lands on the same overtone on the server.

KEY DESIGN DECISION

The browser plays. The server judges.



Never trusts the client

The browser sends settings, never scores. A client-reported accuracy is ignored, and a test proves it.



Rebuilds from scratch

Seeded dirty ingredients and the player settings run through the same pipeline as the reference.



Fair by construction

A dish is claimed atomically, the time bonus runs on the server clock, and locked-tier runs earn no points.

IMPLEMENTATION

How a dish becomes a score

Dish match (server)

$$m = 0.40 \cdot \text{spec} + 0.35 \cdot \text{corr} + 0.25 \cdot \text{SNR}$$

Spectral similarity of dB spectra, correlation, SNR scored from -5 to 25 dB; both dishes on the reference scale, so the seasoning level counts

Final score

$$\text{raw} = 0.72 \cdot m + 0.28 \cdot \text{prep}$$

$$\text{score} = 100 \cdot (\text{raw}/100)^\gamma$$

$\gamma = 1$ / recipe tolerance; stars at 92 / 80 / 66 / 50 / 32

Overall score

$$\text{total} = \text{score} \cdot 10 \cdot d + \min(2 \cdot s, 300) \cdot d$$

$d = 0.8 / 1.0 / 1.25 / 1.5$ from Easy to MasterChef, s = seconds left; the leaderboard ranks this

Live match in each lab

$$0.5 \cdot \max(0, \rho) + 0.5 \cdot (1 - \text{RMSE}/\text{range})$$

ρ = Pearson correlation with the target; instant feedback while you play

IMPLEMENTATION

Hard problems we solved

Mixing curve lost between labs

Carried the 2-D mixing curve through seasoning, marinating and cooking.

Normalising hid seasoning mistakes

Removed per-stage rescaling; dishes are compared on one common scale.

Cart filter started cold

Steady-state periodic response: start-up error of about 100 % of the peak is now under 5 %.

Oven lost everything above $f_s/2$

The equaliser now acts on the full dish; a perfect finish matches the convolved dish at 95–100 %.

The notch amplified the dish $\times 1.17$

Gain normalised at DC or Nyquist, whichever is far from ω_0 .

Score page and leaderboard disagreed

One overall score, computed and stored by the server, shown in both places.

RESULTS

Precision pays, mistakes cost

100%

Perfect cooking matches the reference, on all 10 recipes

95–100%

Final dish after a perfect oven and cart, against the convolved dish

45–72%

Serving with the burnt overtone left in

24–39%

Marinating 20 % too long

Authoritative server scores

Case	Dish score
Server, perfect finish	93–100
Notch left off	44–53
Cart aimed at the aliased reading	41–54
Unstable cart	under 50

RESULTS

Tested end to end

196

frontend tests

Vitest

108

backend tests

pytest

0

type and lint errors

tsc · ESLint

- Browser: signals stay consistent across labs, FFT/IFFT round trips, oven and cart behaviour on every recipe, z-plane systems
- Server: API and security, sessions, scoring, the leaderboard total, finishing stations on every recipe
- End-to-end audits: every recipe played through the real pipeline and the real API

Limitations and next steps

Limitations

- Browser and server build ingredients differently: same parameters, different samples
- The live match is mostly shape-based: seasoning 20 % too strong still shows about 97 %
- Chop Lab aliasing is simulated in the browser; only the server truly decimates
- The beamforming sandbox and direct DSP endpoints are not part of scoring

Next steps

- One shared signal model for browser and server
- Weight amplitude in the live match
- Real frequency folding in the Chop Lab
- A teacher mode with custom recipes and targets

LIVE DEMO

One burger, start to finish

1. Sign in, pick Burger on Medium
2. Wash lettuce and tomato, then mix
3. Season, marinate, grill
4. Oven: safe f_s , notch at 18 Hz
5. Cart: notch at ω_0 , not the aliased reading
6. Score and leaderboard

Thank you

Questions?

Every team member can take questions on any part of the project.